

# One Source, Every Backend: The hanzo-kernel DSL — Thesis and Guided Tour

Hanzo AI — ML Systems

Paper 07 (umbrella) — companion to Paper 06 (*When Does a Kernel Win Surface?*)

## Abstract

The kernel layer of a multi-backend LLM engine is a Cartesian product: the same  $\sim 20$  operations, specialized over  $\sim 22$  quantization formats, hand-written once per GPU backend — five kernel trees and  $\sim 52,000$  lines, and the breeding ground for the divergence bugs that cost the most to chase, because one operation with five implementations has five numeric behaviors. This umbrella paper states the thesis that collapses the product to a sum and gives a guided tour of the six companion papers that make it concrete. The thesis rests on one idea — **fusion is composition**, the functor law made mechanical — extended by three mechanisms that each attack one axis of the product: comptime-specialized autotune (the schedule as a value, not a fork), intrinsic islands (a per-backend idiom held behind a normative oracle arm), and a CPU runtime that is the bit-exact reference for every backend. We describe the facade that ties them together (a first-party namespace over the CubeCL lowering engine, plugged in as a code generator rather than a second runtime), summarize the evidence — portability that costs nothing, a shipped migration that improved performance, an honest structural negative result — and place the work against Triton, XLA, torch.compile, TVM, Halide, Burn, and Mojo. Each fact in this series lives in exactly one paper; here we connect them.

## 1 The thesis: a product that should be a sum

Table 1 is the disease. A multi-backend engine braids three independent concerns into one hand-multiplied product: the operations (the math), the quantization types (the bit layout), and the backends (the lowering). Written by hand, that is  $\sim 20 \times 22 \times 5$  kernels; in the Hanzo repositories it is 52,345 lines across five trees and five shading languages (the ledger and its audit live in the collapse-method paper, [7]).

| Axis        | Count     | What it is                                                        |
|-------------|-----------|-------------------------------------------------------------------|
| Ops         | $\sim 20$ | the math: matvec, matmul, flash-attn, rmsnorm, rope, softmax, GDN |
| Quant types | $\sim 22$ | the bit layout: Q4_K, Q6_K, Q8_0, the IQ codebooks                |
| Backends    | 5         | the lowering: threads, memory, tensor cores, per shading language |

Table 1: Three orthogonal axes, multiplied by hand today. The DSL’s job is to make each a summand.

That product is not merely a maintenance cost. It is where correctness bugs breed: a single quantized model that generates coherent text on one backend and collapses into token repetition on another, from byte-identical weights, because the same operation has a different kernel — and therefore a different numeric behavior — per backend (the bug and its cure are the subject of the

oracle paper, [5]). Uniform numeric behavior across backends cannot be tested *into* five copies; it is a structural property of having one implementation. So the goal is to turn the product into a sum:

$$\underbrace{\text{backends} \times \text{types} \times \text{ops}}_{\text{hand-written today}} \longrightarrow \underbrace{\text{backends} + \text{types} + \text{ops}}_{\text{one source, lowered}}.$$

This is a decompile we already trust in miniature: the ROCm tree writes `qmatvec_core<WTYPE>` — one contraction, the quant type injected as a compile-time parameter. The DSL generalizes exactly that pattern with the *backend* as the other compile-time axis.

## 2 The facade: a namespace, and a code generator

`hanzo-kernel` is a first-party Rust DSL: a kernel is written once, in Rust, and lowered to CUDA, ROCm, Vulkan, and Metal. It is a *facade* over the CubeCL lowering engine — kernel source names only `hanzo_kernel::*`, and the string `cubecl` appears in exactly one line of one `Cargo.toml`. That is “values, not places”: CubeCL is the implementation *value*; `hanzo-kernel` is the stable *namespace* we build against, upgradable or forkable without editing a kernel. The authoring surface uses first-principles names — `#[kernel]` says what a thing is (a GPU kernel), not `cube` (a brand); `Grid` and `Block` name the launch shape directly (Listing 1).

Listing 1: A quantized matvec: one source, no backend in it. The decode (`wd*wq`) and the contraction are the arithmetic each backend hand-writes today. This one function lowers to CUDA, ROCm, Vulkan, Metal, and CPU.

```
use hanzo_kernel::prelude::*;

#[kernel(targets(cuda, metal, vulkan, webgpu, cpu), unchecked)]
pub fn matvec_q8<F: Float>(wd: &Array<F>, wq: &Array<i32>, x: &Array<F>,
    out: &mut Array<F>, #[comptime] k: usize) {
    let row = ABSOLUTE_POS;
    if row < out.len() {
        let (nb, wbase, dbase) = (k / 32, row * k, row * (k / 32));
        let mut acc = F::new(0.0);
        for i in 0..k { acc += wd[dbase + i / 32] * F::cast_from(wq[wbase + i]) * x[i]; }
        out[row] = acc;
    }
}
```

Crucially the DSL is a *code generator*, not a competing GPU runtime. The engine’s Vulkan path is already “load SPIR-V bytes, create a shader module, dispatch,” and the DSL *emits* exactly those bytes: a compiled kernel is a `GLCompute` entry point with buffers on descriptor bindings 0, 1, 2 — byte-for-byte the shape the hand-written `.comp`→`.spv` shaders already take. So the DSL plugs into each backend’s *existing* dispatch machinery; there is no dual runtime and no cross-runtime copy. The one bounded integration step is a three-line SPIR-V post-pass (rename the entry point to `main`; strip the engine’s unused info-uniform; dispatch). The perf primitives the hand-tuned kernels rely on are all reachable and each lowers to the native instruction — `dot` on a vectorized `Line<i8>` → `dp4a`, `cmma` → tensor cores, `SharedMemory` and the `plane.*` subgroup reductions — so portability does not, by construction, force a performance downgrade.

### 3 A guided tour of the mechanisms

The collapse is not a syntactic retyping of kernels; it follows from one algebraic idea applied three ways. Each is developed in full in its own companion paper; here is the map.

**Fusion is composition ([2]).** An elementwise kernel is the lift of a scalar function to every index, and such lifts obey the functor law  $\text{map}(g) \circ \text{map}(f) = \text{map}(g \circ f)$ . Read right-to-left, that law *is* fusion: one kernel computing the composite, no intermediate array. Reduce and Scan are algebraic *fences* where the pattern changes, decidable from the op’s class rather than a pattern matcher. This is the theory core: the fusibility of two ops is a total function of their type, one fused-interpreter kernel lowers any composed chain to a single launch, and the same algebra lifts from a line to an arbitrary DAG — draining, for instance, a per-layer Gated-DeltaNet chain from 9 kernel launches to 5, bit-exact.

**The schedule is a value ([3]).** The last place a backend forks a kernel is the schedule — tile size, vector width, unroll, block dimension. A fork is exactly what reintroduces the divergence bug. Autotune folds the fork into the one source by exposing the schedule as compile-time knobs the engine monomorphizes; a small variant set is timed per device and the winner cached to disk. Because the knobs never touch the arithmetic, every variant is byte-identical — the tuner buys speed and can never spend correctness.

**Intrinsic islands ([4]).** When one inner line wants a target-specific instruction, you do not fork the kernel; you write a scoped, target-gated block *inside* it. Selection is a compile-time `match` on a tag the launch layer fills in, so exactly one arm is lowered per backend. The `default` arm is *normative* — the CPU always takes it, and every accelerated arm is proven byte-for-byte equal to it on the CPU oracle before it ships. An escape hatch that cannot escape correctness.

**One oracle, every backend ([5]).** The foundation under all three: the CPU runtime is the single bit-exact reference for every backend. Every op’s device code mirrors a plain-Rust reference line-for-line; because the CPU runs pointwise ops in program order, its result is the definition, and a GPU differs only by bounded reduction-order rounding. The law is enforced nightly by a three-backend GPU CI on self-hosted NVIDIA, AMD, and Apple machines — so “the same op has the same numbers everywhere” is a checked invariant, and the repetition collapse becomes an unreachable state rather than a bug to fix.

**Where the product won’t collapse ([6]).** Honesty is part of the thesis. Block-quantized tensor-core MMQ — the kernel that governs prefill — does *not* collapse through the portable API: its accumulator fragment is opaque, forcing a per-block scale round-trip that caps it near one-tenth of hand-tuned (proven sync-bound because better tiling is *slower*). The viable route is per-backend anyway. So MMQ stays hand-written behind exactly the island seam built for it — a named boundary, not a quiet exclusion.

**Verify, then delete ([7]).** The migration method ties it together: a hand-written kernel is retired only when its single-source twin is both bit-exact against the oracle and at least as fast on a kernel-only benchmark, cell by cell through a per-(`op,type,backend`) registry, so the engine is faster-or-equal at every step. Cells have already flipped on `main` — including a Vulkan `add_rmsnorm` twin that was  $7.1\times$  *faster* than the shader it replaced and bit-exact.

## 4 Evidence, in one breath

The claims the tour makes are backed by measurement, detailed in the companion papers: the op library is bit-exact against the CPU oracle and dispatches bit-exact through real Vulkan, Metal, and ROCm runtimes (rms\_norm on gfx1151 to  $2.43 \times 10^{-7}$ ; [5]); a single-source **dp4a** matvec runs at 118% of the hand-tuned kernel it replaces (196 GB/s on gfx1151; [7]); a shipped migration was  $7.1\times$  faster *and* bit-exact ([7]); the autotune variants are byte-identical with a per-device cached winner ([3]); and the one op that resists is reported in full as a structural negative result ([6]). Portability costs nothing where it counts, the collapse is real and reversible, and the boundary is drawn honestly.

## 5 Related work

Portable kernel authoring and kernel fusion are both well-trodden; our specific claim is narrow, and we state it precisely. **Triton** gives a tile-level Python DSL, but fusion is the programmer’s job or a larger stack’s; it has no notion of fusion as an algebraic law over an op class. **XLA** fuses HLO graphs by producer-consumer heuristics and a cost model; **torch.compile**/TorchInductor captures a graph and fuses by a scheduler’s heuristics — both fuse by *analysis over a graph*, not by a predicate that is a function of the op’s type. **Halide** and **TVM** pioneered separating the *compute* from the *schedule* and autotuning the latter; this is the nearest neighbor of our autotune mechanism ([3]), and we claim no novelty for schedule/compute separation itself — what differs is that our schedule knobs are compile-time monomorphizations of one kernel proven byte-identical, so autotuning cannot perturb numerics. **Burn**’s **burn-fusion** is the closest relative to our fusion engine: it records an op stream and JITs fused elementwise kernels over the *same* CubeCL substrate, but it traces Burn tensor ops with its own cut/epilogue policy, coupling to Burn’s tensor abstraction; ours is the functor law over an op *class*, one predicate, on our own tensor type, composing with our hand-tuned peak kernels. **Mojo** is a language for kernels, not a fusion-as-composition model. Across all of these we are unaware of a system that presents kernel fusion *as* morphism composition — the functor law as the mechanical rule, Reduce/Scan as the algebraic fences — anchored by a bit-exact CPU oracle as the definitional semantics. That framing, and the three mechanisms it organizes, is the contribution; the substrate (CubeCL) and the individual ideas (portable IR, autotuning, oracle testing) are prior art we build on and cite.

## 6 Conclusion

The kernel layer is a Cartesian product of backends, quantization types, and operations, hand-written in five shading languages, and it is the breeding ground for the divergence bugs that cost the most to chase. **hanzo-kernel** collapses that product to a sum on one idea — fusion is composition ([2]) — extended by three mechanisms that each attack an axis: autotune folds the schedule fork into byte-identical comptime knobs ([3]); islands hold the rare per-backend idiom behind a normative, CPU-checkable arm ([4]); and the CPU runtime is the single bit-exact oracle every backend is measured against, nightly ([5]). Portability costs nothing where it counts, the collapse is real and reversible via a verify-then-delete method ([7]), and — stated as plainly as the wins — one op resists and stays hand-written behind an island ([6]). Read the six companion papers for the depth; this one is the map.

## References

- [1] Hanzo AI. *One Source, Every Backend: The hanzo-kernel DSL*. Paper 07 (this umbrella).
- [2] Hanzo AI. *Fusion Is Composition: Kernel Fusion as the Functor Law*. Paper 07.1.
- [3] Hanzo AI. *The Schedule Is a Value: Comptime-Specialized Autotuning*. Paper 07.2.
- [4] Hanzo AI. *Intrinsic Islands: An Oracle-Anchored Escape Hatch*. Paper 07.3.
- [5] Hanzo AI. *One Oracle, Every Backend: The CPU Bit-Exactness Law*. Paper 07.4.
- [6] Hanzo AI. *When the Product Won't Collapse: An int8 Tensor-Core MMQ Negative Result*. Paper 07.5.
- [7] Hanzo AI. *Verify, Then Delete: Collapsing a Kernel Cartesian Product*. Paper 07.6.